

UNIVERSIDADE ESTADUAL DO MARANHÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE ENGENHARIA DA COMPUTAÇÃO
ESTRUTURA DE DADOS AVANÇADA

TEORIA DA COMPLEXIDADE

Márcio Honorindo Freires Citó

São Luís

2025

TEORIA DA COMPLEXIDADE

1. INTRODUÇÃO

Para esse trabalho, escolhi a linguagem C e os algoritmos Bubblesort, Selectionsort e Mergesort por questão de familiaridade, foram algoritmos vistos na disciplina de estrutura de dados básico, logo, também já estavam prontos. Outro ponto foram as suas diferentes complexidades em cada caso:

Tabela 1. Algoritmos de ordenação

Algoritmo	Melhor	Médio	Pior	Memória
Bubblesort	n	n^2	n^2	1
Selectionsort	n^2	n^2	n^2	1
Mergesort	$n \log n$	$n \log n$	$n \log n$	n

Como é mostrado na tabela, feita a partir de dados do site wikipedia, o outro motivo para a escolha desses algoritmos foi por terem diferentes níveis de complexidade e memória entre si, sendo possível de realizar uma comparação mais distinta entre eles.

O bubblesort compara os elementos de um vetor de forma decrescente, que funciona de modo a comparar a posição atual com a posição seguinte, se o valor atual for maior que o valor na posição posterior, eles trocam de posição, se não é passado para a posição seguinte, ao chegar no fim da lista, o algoritmo retorna para o início e faz novamente o processo até que todo o vetor esteja ordenado, apesar de ser o algoritmo de ordenação mais simples é também o menos eficiente, sem ter uso prático no mundo real, sendo mais utilizado para ensinamento acadêmico.

O selectionsort percorre um vetor e seleciona seu menor elemento, e logo após coloca esse elemento na posição inicial do vetor, depois se repete o processo até que toda a lista esteja ordenada. Assim como o bubblesort, é necessário para cada item do vetor percorrer ele inteiro, então serão precisos dois loops, por isso a sua alta complexidade e o alto tempo de execução para um grande número de dados.

O mergesort é um algoritmo recursivo que utiliza do método de divisão por conquista, ele consiste em dividir o vetor pela metade repetidas vezes, até obter um vetor de um elemento para depois fazer a junção com o vetor seguinte de forma ordenada até que toda a lista esteja ordenada, apesar de ser um algoritmo muito mais eficiente que os outros citados, tem um gasto de memória maior pelo fato do método utilizado sempre criar vetores novos.

2. DESENVOLVIMENTO

```

19 void Bubblesort(int v[], int n){
20     int aux;
21     for (int i=0; i<n-1; i++){ //O(n-1)
22         for (int j=0; j<n-i-1; j++){ //O(n-i-1)
23             if (v[j]>v[j+1]){ //O(n^2)
24                 aux = v[j]; //O(n^2)
25                 v[j] = v[j+1]; //O(n^2)
26                 v[j+1] = aux; //O(n^2)
27             }
28         }
29     }
30 }

```

Figura 1. Código Bubblesort

Na figura acima é mostrado o código bubblesort e o cálculo de complexidade em cada linha. Considerando que na linha 21 tem um laço de repetição que se inicia em 0 e vai até $n-1$, a sua complexidade é de $O(n-1)$ que é igual a $O(n)$, agora na linha seguinte possui outro laço com início 0 até $n-i-1$, logo, a sua complexidade é de $O(n-i-1)$, porém, como i varia de 0 a $n-1$, o cálculo de complexidade é resultado de um somatório, visto que, na primeira iteração para $i = 0$, o j roda $n-1$ vezes, para $i = 1$, o j roda $n-2$ vezes, e assim por diante, até o ponto em que, para i ser menor do que $n-1$, i precisa ser igual a $n-2$, nessa iteração j será $n-(n-2)-1 = 1$, logo $j < 1$, então j roda 1 vez. Isso explica que a quantidade de iterações de j é um somatório que varia de $n-1$ até 1 com uma razão de -1 com $n-1$ elementos, logo, pode-se usar a fórmula do somatório de elementos de uma progressão aritmética:

$$Sk = \frac{k(a_1 + ak)}{2}$$

Em que, k é a quantidade de elementos, a_1 é o primeiro elemento e ak é o último, no caso do algoritmo, $k = n-1$ iterações/elementos, $a_1 = n-1$, $ak = 1$, isso resulta em:

$$S = \frac{(n-1)(n-1+1)}{2} \rightarrow \frac{n(n-1)}{2}$$

Então a complexidade é de $O\left(\frac{n(n-1)}{2}\right)$ porém, seguindo as regras de propriedades e operações para complexidade, em que se deve desconsiderar as constantes, o resultado final é de $O(n^2)$ para a linha 22, as demais linhas a partir do `if` estão dentro do segundo laço, então no pior caso, elas serão executadas em todas as iterações de j , resultando também em uma complexidade de $O(n^2)$.

```

32 //SELECTION SORT
33 int Selmin(int *v,int i, int n){ //O(n-1)
34     int k=i; //O(n-1)
35     for(int j=i+1;j<n;j++){ //O(n^2)
36         if(v[j]<v[k]){ //O(n^2)
37             k=j; //O(n^2)
38         }
39     }
40     return k; //O(n-1)
41 }
42
43 void selectionsort(int *v, int n){
44     int k;
45     for(int i=0;i<n-1;i++){ //O(n-1)
46         k=Selmin(v,i,n); //O(n-1)
47         int x; //O(n-1)
48         x = v[i]; //O(n-1)
49         v[i]=v[k]; //O(n-1)
50         v[k] = x; //O(n-1)
51     }
52 }

```

Figura 2. Código Selectionsort

O código principal começa com um laço de repetição em que a variável i varia de 0 até $n-1$, ou seja, tem complexidade $O(n-1)$, que é igual a $O(n)$, logo abaixo, ele chama a função `Selmin`, que vai fazer a seleção do menor número no vetor. Essa função é iniciada por outro laço de repetição, que se inicia em $i+1$ até n , isso nos coloca em uma situação semelhante ao laço duplo analisado no Bubblesort, quando $i = 0$, j varia de 1 até $n-1$, para $i = 1$, j varia de 2 até $n-1$, para $i = 2$, j varia de 3 até $n-1$ e assim até $i = n-2$. Formando assim, outro somatório para ser resolvido utilizando a fórmula do somatório de elementos de uma progressão aritmética, em que, o primeiro elemento é $n-1$, que é a quantidade de iterações de j , e o último é 1 para $n-1$ iterações totais de i , resultando em:

$$S = \frac{(n-1)(n-1+1)}{2} \rightarrow \frac{n(n-1)}{2}$$

Que é o mesmo observado no Bubblesort, logo também tem complexidade de $O(n^2)$, e por isso, as outras linhas 36 e 37 também vão ter a mesma complexidade visto que vão executar a mesma quantidade de vezes que j , já a linha 40 está fora do laço de repetição de j , mas está dentro do laço de repetição de i , então por isso possui complexidade de $O(n-1)$ que é igual a $O(n)$, assim como todas as outras demais linhas que estão dentro do laço de repetição do i : linha 47, 48, 49 e 50.

```

88 void mergeSort(int v[], int l, int r) {
89     if (l < r) { //O(1)
90         int m = l + (r - l) / 2; //O(1)
91         mergeSort(v, l, m); //T([n/2])
92         mergeSort(v, m + 1, r); //T([n/2])
93         merge(v, l, m, r); //O(n)
94     }
95 }

```

Figura 3. Código Mergesort

Considerando $T(n)$ como o consumo de tempo máximo quando o número de elementos $n = r - l + 1$, e que, devido ao mergesort ser o um algoritmo recursivo que sempre parte o problema em dois pela linha 90, o consumo de tempo quando ele faz uma chamada recursiva sempre cai pela metade, logo as linhas 91 e 92 possuem complexidade $T([n/2])$, no total ficando, $T(n) = T([n/2]) + T([n/2]) + O(n) + O(2)$, que pode ser simplificado para $T(n) = 2 T([n/2]) + O(n)$. Como o problema sempre é dividido pela metade, pode-se supor que $n = 2^k$ então, $k = \log_2 n$, substituindo n por 2^k na fórmula se obtém: $T(2^k) = 2T(2^{k-1}) + 2^k$, a partir disso, deve-se substituir o $T(n)$ pelo novo valor 2^{k-1} chegando ao resultado de $2(2T(2^{k-2}) + 2^{k-1}) + 2^k$ fazendo um looping até que o expoente de 2 seja 0 para obter o valor de $T(1)$: $2(2(\dots(2T(1)+2^2)+ 2^3)+\dots)+ 2^{k-1})+ 2^k$, simplificando se torna: $(k-1)2^k + 2^k = k2^k$, agora voltando ao início onde $n = 2^k$ e $k = \log_2 n$, então por fim: $T(n) = n \log_2 n$, ou seja, sua complexidade final é de $O(n \log_2 n)$.

3. RESULTADOS

A partir do arquivo de saída, é possível observar os resultados da comparação entre os algoritmos. Primeiramente, percebe-se que para um n pequeno não é possível determinar um tempo de execução por ser um tempo também pequeno, porém conforme vai aumentando, o tempo de execução também cresce com um certo padrão que difere entre os algoritmos, o bubblesort e selectionsort possuem padrões semelhantes e tempos de execução semelhante para cada caso, por outro lado, o mergesort se mostrou bem mais eficiente independentemente do caso ou do número de elementos. O padrão percebido no bubble e selection é que a cada 10 elementos que aumentam, o tempo de execução aumenta em x100, já para o merge que quando é aumentado 10 elementos, o seu tempo de execução aumenta em x10, porém, para ambos os casos existem variações imprevisíveis no tempo de execução, mas no geral, a ordem de grandeza não muda.

Tabela 2. Comparação entre o tempo de execução

Algoritmo	n	Caso	Tempo(s)
Selectionsort	10000	melhor	0.1270
Selectionsort	100000	melhor	12.939
Selectionsort	1000000	melhor	1302.323

Mergesort	100000	melhor	0.045
Mergesort	1000000	melhor	0.435
Mergesort	10000000	melhor	4.267

Na tabela é mostrado o padrão citado anteriormente e a comparação da eficiência do selectionsort para o mergesort, enquanto para 10^6 elementos do selectionsort deram 1302 segundos, que é aproximadamente 22 minutos, o mergesort para 10^7 elementos demorou pouco mais de 4 segundos. Para esse trabalho não foi realizado a execução do bubble e selection para a quantidade de 10^7 elementos, pois, seguindo a lógica já citada, o tempo de execução seria aproximadamente 130.000 segundos, que daria 36 horas, algo que foi inviável no momento considerando todos os casos. Outro ponto importante a ser ressaltado, era esperado que o tempo de execução do melhor caso para o pior caso no bubblesort fossem bem diferentes, mas não foi o que aconteceu, eles praticamente se mantiveram iguais para cada n .

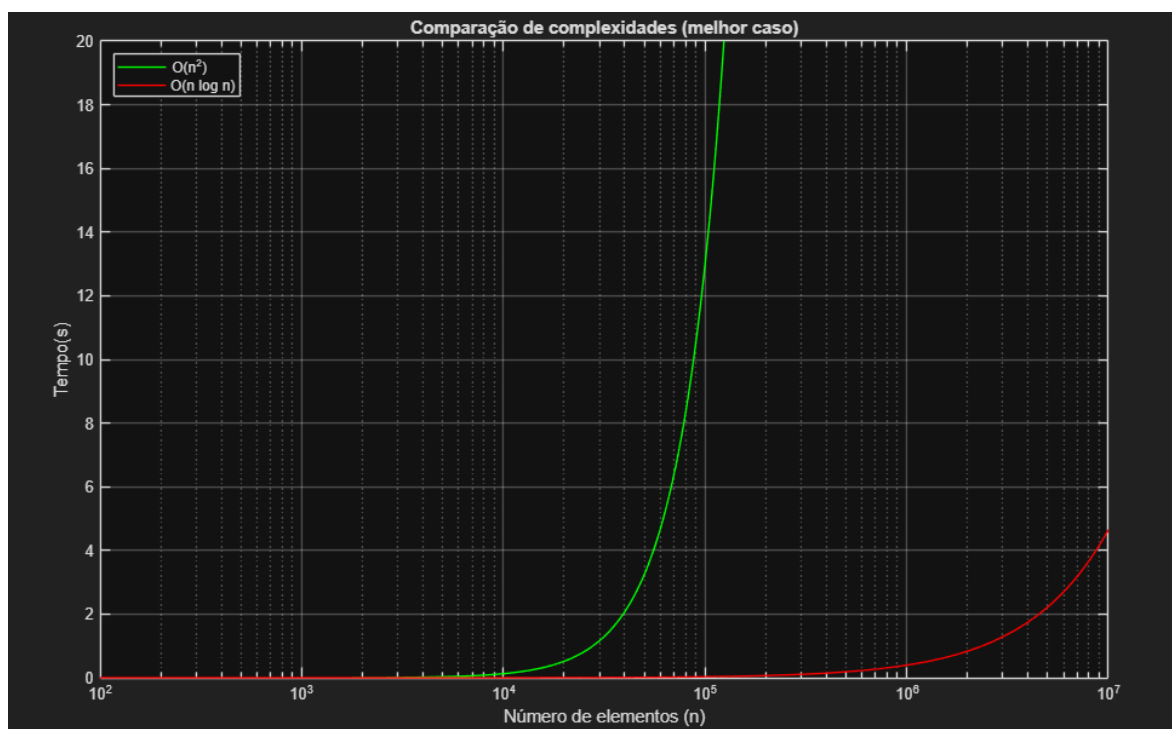


Figura 4. Gráfico comparativo (tempo x n)

Nesse gráfico é mostrado o comparativo entre o selectionsort, que é semelhante ao bubble, e mergesort, em relação ao tempo de execução e o número de elementos. Ao analisar o gráfico, percebe-se que para um número de elementos pequeno inicialmente, o tempo é praticamente nulo, porém ao aumentar, o tempo cresce exponencialmente a partir de um determinado n , no caso da complexidade $O(n^2)$ a partir de 10.000 elementos, ele começa a crescer de forma explosiva, enquanto o de complexidade $O(n \log_2 n)$ cresce somente aos 100.000 elementos e de forma mais controlada.

```

1      n = 10:100:1e7;
2
3      c1 = 1.3.*10.^(-9);
4      c2 = 2.*10.^(-8);
5
6      f1 = c1.*n.^2;
7      f2 = c2.*n.*log2(n);
8
9      plot(n, f1, 'g', n, f2, 'r');
10     legend('O(n^2)', 'O(n log n)', 'Location', 'northwest');
11     xlabel('Número de elementos (n)');
12     ylabel('Tempo(s)');
13     title('Comparação de complexidades (melhor caso)');
14     grid on;
15     xlim([1e2 1e7])
16     ylim([0 20])
17     set(gca, 'XScale', 'log')

```

Figura 5. Código da Figura 4

Acima é mostrado o código em MATLAB usado para gerar a Figura 4, a partir da complexidade dos algoritmos e de suas complexidades foi possível determinar constantes para se aproximar ao tempo de execução, no caso de $O(n^2)$ foi utilizado $T(n) = c1 \cdot n^2$, dessa forma, considerando o tempo de 1300 segundos para 1.000.000 no selectionsort, foi feita a substituição, o que resultou em $c1 = 1.3 \cdot 10^{-9}$. Para o mergesort foi utilizado a mesma lógica, porém com resultado diferente pelo fato de ser um logaritmo, para $O(n \log_2 n)$ foi utilizado $T(n) = c2 \cdot n \cdot \log_2 n$, então, utilizando uma média de 0.4 segundos para 1.000.000 chegou-se ao resultado de $c2 = 2 \cdot 10^{-8}$, que não consegue prever perfeitamente o tempo para uma quantidade maior ou menor de elementos, porém é possível analisar a sua aproximação.

4. CONCLUSÃO

No geral, foi possível observar e comparar a diferença entre os algoritmos de ordenação de modo satisfatório, apesar de ter alguns pontos fora do esperado, como o caso do bubblesort não apresentar diferença no tempo de execução entre o seu melhor e pior caso, porém a diferença entre o mergesort e selectionsort foi nítida e coerente.

5. REFERÊNCIAS

https://en.wikipedia.org/wiki/Sorting_algorithm

https://www-geeksforgeeks-org.translate.goog/bubble-sort-algorithm/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

https://deinfo.uepg.br/~alunoso/2022/AEP/SELECTION_SORT/

Slides 12, 13, 14 e 15 da disciplina